

Plano de Aprendizado: Go (Golang) — Fases 4 a 10

Continuação do plano. O projeto `taskctl` continua evoluindo. Ao final da Fase 10, você tem um sistema pronto pra portfólio e entrevista.

Fase 4 — Goroutines e Concorrência

Objetivo: entender o diferencial do Go. Goroutines são threads leves gerenciadas pelo runtime do Go — você cria milhares sem problema. É o que faz Go brilhar em sistemas de alta concorrência (e o que as vagas pedem).

4.1 — Goroutines Básicas

O que você aprende: como rodar funções em paralelo. Uma goroutine é uma função executada concorrentemente — basta colocar `go` antes da chamada.

```
package main

import (
    "fmt"
    "time"
)

func processarTarefa(id int, titulo string) {
    fmt.Printf("Iniciando tarefa %d: %s\n", id, titulo)

    time.Sleep(2 * time.Second) // Simula trabalho pesado

    fmt.Printf("Tarefa %d concluída: %s\n", id, titulo)
}

func main() {
```

```

// SEM goroutines — sequencial (6 segundos total)

// processarTarefa(1, "Backup")

// processarTarefa(2, "Relatório")

// processarTarefa(3, "Email")

// COM goroutines — concorrente (2 segundos total)

go processarTarefa(1, "Backup")

go processarTarefa(2, "Relatório")

go processarTarefa(3, "Email")

// Problema: main() termina antes das goroutines completarem

// Solução temporária (ruim pra produção):

time.Sleep(3 * time.Second)

fmt.Println("Todas concluídas")

}

```

Conceito crucial: goroutines são absurdamente baratas. Uma thread do OS usa ~1MB de stack. Uma goroutine começa com ~2KB. Você pode ter centenas de milhares rodando ao mesmo tempo. É por isso que Go é a linguagem de escolha pra microserviços e sistemas de rede.

Comparação: | Linguagem | Concorrência | Custo | |---|---|---| | PHP | Processos separados (php-fpm) | Pesado | | Python | asyncio / threads (GIL limita) | Médio, limitado | | Go | Goroutines (M:N scheduling) | Ultra leve |

4.2 — WaitGroup (Esperando Goroutines Terminarem)

O que você aprende: `sync.WaitGroup` é a forma correta de esperar goroutines concluírem — substitui o `time.Sleep` que usamos acima.

```
package main
```

```
import (  
  
    "fmt"  
  
    "sync"  
  
    "time"  
  
)  
  
func processarTarefa(id int, titulo string, wg *sync.WaitGroup) {  
  
    defer wg.Done() // Sinaliza "terminei" quando a função retorna  
  
    fmt.Printf("Processando tarefa %d: %s\n", id, titulo)  
  
    time.Sleep(time.Duration(id) * time.Second) // Simula tempos diferentes  
  
    fmt.Printf("Tarefa %d concluída\n", id)  
  
}  
  
func main() {  
  
    var wg sync.WaitGroup  
  
    tarefas := map[int]string{  
  
        1: "Backup banco",  
  
        2: "Gerar relatório",  
  
        3: "Enviar emails",  
  
    }  
  
    for id, titulo := range tarefas {  
  
        wg.Add(1) // Incrementa o contador ANTES de lançar a goroutine  
  
        go processarTarefa(id, titulo, &wg)
```

```
}

wg.Wait() // Bloqueia até o contador chegar a zero

fmt.Println("Todas as tarefas concluídas")

}
```

Padrão importante:

1. `wg.Add(1)` — antes de lançar a goroutine
 2. `defer wg.Done()` — primeira linha da goroutine
 3. `wg.Wait()` — onde você quer esperar tudo terminar
-

4.3 — Channels (Comunicação entre Goroutines)

O que você aprende: channels são o mecanismo de comunicação entre goroutines. O lema do Go é: "Não comunique compartilhando memória; compartilhe memória comunicando." Channels são a implementação disso.

```
package main

import "fmt"

func main() {

    // Criar channel (tipado — só transporta um tipo)

    mensagens := make(chan string)

    // Enviar valor pro channel (em uma goroutine)

    go func() {

        mensagens <- "Tarefa 1 concluída" // <- envia

    }()

    // Receber valor do channel (bloqueia até receber)

    msg := <-mensagens // <- recebe
```

```

fmt.Println(msg)

// CHANNEL BUFFERED — tem capacidade, não bloqueia até encher

fila := make(chan string, 3) // buffer de 3

fila <- "Tarefa A"

fila <- "Tarefa B"

fila <- "Tarefa C"

// fila <- "Tarefa D" // Bloquearia — buffer cheio

fmt.Println(<-fila) // Tarefa A (FIFO)

fmt.Println(<-fila) // Tarefa B

}

```

Aplicação real no taskctl — processamento de tarefas em background:

```

package main

import (
    "fmt"
    "time"
)

type Resultado struct {
    TarefaID int
    Sucesso  bool
    Mensagem string
}

```

```

func processarEmBackground(id int, titulo string, resultados chan<- Resultado) {

    // chan<- = channel de envio apenas (send-only)

    fmt.Printf("Processando: %s\n", titulo)

    time.Sleep(time.Second)

    resultados <- Resultado{

        TarefaID: id,

        Sucesso: true,

        Mensagem: fmt.Sprintf("%s' processada com sucesso", titulo),

    }

}

func main() {

    resultados := make(chan Resultado, 10)

    tarefas := []string{"Backup", "Relatório", "Sync", "Notificação"}

    // Lançar todas em paralelo

    for i, titulo := range tarefas {

        go processarEmBackground(i+1, titulo, resultados)

    }

    // Coletar resultados

    for i := 0; i < len(tarefas); i++ {

        r := <-resultados

        fmt.Printf("Resultado tarefa %d: %s (sucesso: %t)\n",

            r.TarefaID, r.Mensagem, r.Sucesso)
    }
}

```

```
}  
  
}
```

4.4 — Select (Multiplexação de Channels)

O que você aprende: `select` permite ouvir múltiplos channels ao mesmo tempo — quem chegar primeiro ganha. É como um `switch` pra channels.

```
package main  
  
import (  
  
    "fmt"  
  
    "time"  
  
)  
  
func main() {  
  
    rapido := make(chan string)  
  
    lento := make(chan string)  
  
    go func() {  
  
        time.Sleep(100 * time.Millisecond)  
  
        rapido <- "resposta rápida"  
  
    }()  
  
    go func() {  
  
        time.Sleep(2 * time.Second)  
  
        lento <- "resposta lenta"  
  
    }()
```

```
// Select com timeout — padrão comum em APIs

select {

case msg := <-rapido:

    fmt.Println("Rápido:", msg)

case msg := <-lento:

    fmt.Println("Lento:", msg)

case <-time.After(1 * time.Second):

    fmt.Println("Timeout: nenhuma resposta a tempo")

}

}
```

Uso real: `select` é o pattern de timeout em chamadas de API, consultas ao banco, e qualquer operação que não pode travar pra sempre.

4.5 — Mutex (Protegendo Dados Compartilhados)

O que você aprende: quando múltiplas goroutines acessam o mesmo dado ao mesmo tempo, pode dar race condition. `sync.Mutex` trava o acesso — só uma goroutine por vez.

```
package main

import (

    "fmt"

    "sync"

)

// Contador seguro pra uso concorrente
```



```
type ContadorTarefas struct {  
  
    mu    sync.Mutex  
  
    total int  
  
    feitas int  
  
}  
  
func (c *ContadorTarefas) Incrementar() {  
  
    c.mu.Lock()    // Trava — ninguém mais entra  
  
    defer c.mu.Unlock() // Destrava quando a função retorna  
  
    c.total++  
  
}  
  
func (c *ContadorTarefas) Concluir() {  
  
    c.mu.Lock()  
  
    defer c.mu.Unlock()  
  
    c.feitas++  
  
}  
  
func (c *ContadorTarefas) Stats() (int, int) {  
  
    c.mu.Lock()  
  
    defer c.mu.Unlock()  
  
    return c.total, c.feitas  
  
}  
  
func main() {  
  
    contador := &ContadorTarefas{}
```

```
var wg sync.WaitGroup

// 100 goroutines incrementando ao mesmo tempo

for i := 0; i < 100; i++ {

    wg.Add(1)

    go func() {

        defer wg.Done()

        contador.Incrementar()

        contador.Concluir()

    }()

}

wg.Wait()

total, feitas := contador.Stats()

fmt.Printf("Total: %d | Feitas: %d\n", total, feitas)

// Sem Mutex: números imprevisíveis (race condition)

// Com Mutex: sempre 100 e 100

}
```

Quando usar Mutex vs Channel:

- **Mutex** → proteger dados compartilhados (contadores, caches, maps)
 - **Channel** → comunicação entre goroutines (enviar/receber valores)
-

4.6 — Worker Pool Pattern

O que você aprende: padrão de produção pra processar muitas tarefas com um número fixo de workers. É exatamente o que um sistema de filas faz (você já trabalha com isso no GOINFRA via Redis/workers).

```
package main

import (
    "fmt"
    "sync"
    "time"
)

type Job struct {
    ID    int
    Titulo string
}

type Result struct {
    JobID    int
    Sucesso bool
}

func worker(id int, jobs <-chan Job, results chan<- Result, wg *sync.WaitGroup) {
    defer wg.Done()

    for job := range jobs { // Fica ouvindo até o channel fechar
        fmt.Printf("Worker %d processando job %d: %s\n", id, job.ID, job.Titulo)
```

```

        time.Sleep(500 * time.Millisecond) // Simula trabalho

        results <- Result{JobID: job.ID, Sucesso: true}
    }

    fmt.Printf("Worker %d encerrado\n", id)
}

func main() {

    numWorkers := 3

    jobs := make(chan Job, 10)

    results := make(chan Result, 10)

    // Iniciar workers

    var wg sync.WaitGroup

    for i := 1; i <= numWorkers; i++ {

        wg.Add(1)

        go worker(i, jobs, results, &wg)

    }

    // Enviar jobs

    tarefas := []string{

        "Processar CSV", "Gerar PDF", "Enviar email",

        "Sync banco", "Backup", "Notificar",

        "Limpar cache", "Reindexar", "Validar dados",

    }

    for i, titulo := range tarefas {

```

```
    jobs <- Job{ID: i + 1, Titulo: titulo}

}

close(jobs) // Sinaliza que não tem mais jobs

// Esperar workers terminarem em outra goroutine e fechar results

go func() {

    wg.Wait()

    close(results)

}()

// Coletar resultados

for r := range results {

    fmt.Printf("Job %d concluído (sucesso: %t)\n", r.JobID, r.Sucesso)

}

}
```

Conexão com seu trabalho atual: esse pattern é equivalente ao que você faz no GOINFRA com Redis + workers de fila (Laravel Jobs/Horizon). A diferença é que em Go, os workers são goroutines (extremamente leves) em vez de processos separados.

Fase 5 — Testes

Objetivo: Go tem um framework de testes embutido (**testing**) — não precisa instalar nada. Aprender a testar é requisito de qualquer vaga Pleno.

5.1 — Testes Unitários Básicos

O que você aprende: arquivos de teste terminam com `_test.go`. Funções de teste começam com `Test`. Rodam com `go test`.

```
// repository/tarefa_repo_test.go

package repository

import "testing"

// Testar a validação de título

func TestValidarTitulo(t *testing.T) {

    // Caso: título válido

    if !validarTitulo("Aprender Go") {

        t.Error("esperava true para título válido")

    }

    // Caso: título vazio

    if validarTitulo("") {

        t.Error("esperava false para título vazio")

    }

}

// Função sendo testada (no mesmo package)

func validarTitulo(titulo string) bool {

    return titulo != ""

}

go test ./...      # Roda todos os testes do projeto
```

```
go test -v ./...      # Verbose — mostra cada teste
```

```
go test -run TestValidar # Roda só testes que batem com o pattern
```

```
go test -cover ./...  # Mostra cobertura de testes
```

5.2 — Table-Driven Tests (Padrão do Go)

O que você aprende: o idioma mais comum de testes em Go. Define uma tabela de cenários e itera sobre eles. Evita copiar/colar o mesmo teste com valores diferentes.

```
func TestValidarTitulo_TableDriven(t *testing.T) {

    testes := []struct {

        nome    string // Nome do cenário

        titulo  string

        esperado bool

    }{

        {"título válido", "Aprender Go", true},

        {"título vazio", "", false},

        {"título com espaços", "  ", true}, // Depende da regra de negócio

        {"título longo", "A" + string(make([]byte, 500)), true},

    }

    for _, tt := range testes {

        t.Run(tt.nome, func(t *testing.T) {

            resultado := validarTitulo(tt.titulo)

            if resultado != tt.esperado {
```

```

        t.Errorf("validarTitulo(%q) = %v, esperado %v",
            tt.titulo, resultado, tt.esperado)
    }

    })

}

}

```

Por que table-driven:

- Adicionar um cenário novo = 1 linha na tabela
- `t.Run` cria sub-testes nomeados (aparecem separados no output)
- É o padrão que todo code review em Go espera ver

5.3 — Testando HTTP Handlers com `httptest`

O que você aprende: Go tem um pacote embutido pra testar handlers HTTP sem subir servidor real.

```

package handler

import (
    "net/http"
    "net/http/httptest"
    "strings"
    "testing"
)

func TestCriarTarefa_SemTitulo(t *testing.T) {
    // Criar request fake

```



```

body := strings.NewReader(`{"titulo": ""}`)

req := httptest.NewRequest(http.MethodPost, "/tarefas", body)

req.Header.Set("Content-Type", "application/json")

// Criar response recorder (grava a resposta)

rec := httptest.NewRecorder()

// Chamar o handler diretamente (sem servidor HTTP)

h := NovoTarefaHandler(repoFake) // repo fake/mock

h.Criar(rec, req)

// Verificar

if rec.Code != http.StatusBadRequest {

    t.Errorf("esperava status 400, recebeu %d", rec.Code)

}

}

func TestListarTarefas_Vazio(t *testing.T) {

    req := httptest.NewRequest(http.MethodGet, "/tarefas", nil)

    rec := httptest.NewRecorder()

    h := NovoTarefaHandler(repoFake)

    h.Listar(rec, req)

    if rec.Code != http.StatusOK {

        t.Errorf("esperava status 200, recebeu %d", rec.Code)

    }

}

```

```
// Verificar se retornou array JSON vazio

expected := "[]"

if strings.TrimSpace(rec.Body.String()) != expected {

    t.Errorf("esperava %s, recebeu %s", expected, rec.Body.String())

}

}
```

5.4 — Interfaces pra Mocks

O que você aprende: como criar mocks usando interfaces — sem biblioteca externa. Como interfaces em Go são implícitas, qualquer struct que implemente os mesmos métodos serve de mock.

```
// Definir interface do repository

type TarefaStore interface {

    Listar(ctx context.Context) ([]Tarefa, error)

    Criar(ctx context.Context, titulo string) (*Tarefa, error)

    BuscarPorID(ctx context.Context, id int) (*Tarefa, error)

    Atualizar(ctx context.Context, id int, titulo *string, feita *bool) (*Tarefa, error)

    Deletar(ctx context.Context, id int) error

}

// Handler agora depende da interface, não do struct concreto

type TarefaHandler struct {

    repo TarefaStore // interface em vez de *TarefaRepository

}
```

```

// Mock pra testes

type MockTarefaStore struct {

    tarefas []Tarefa

}

func (m *MockTarefaStore) Listar(ctx context.Context) ([]Tarefa, error) {

    return m.tarefas, nil

}

func (m *MockTarefaStore) Criar(ctx context.Context, titulo string) (*Tarefa, error) {

    t := Tarefa{ID: len(m.tarefas) + 1, Titulo: titulo}

    m.tarefas = append(m.tarefas, t)

    return &t, nil

}

// ... implementar os outros métodos

// No teste:

func TestListar_ComMock(t *testing.T) {

    mock := &MockTarefaStore{

        tarefas: []Tarefa{

            {ID: 1, Titulo: "Teste"},

        },

    }

    h := &TarefaHandler{repo: mock}

```

```
// ... testar normalmente  
  
}
```

Ponto-chave: em Go, você define a interface **no consumidor** (no handler), não no produtor (no repository). Isso é o oposto de Java/PHP. O handler diz "preciso de algo que saiba Listar e Criar" — qualquer struct que tenha esses métodos serve.

5.5 — Benchmarks

O que você aprende: Go tem benchmarks embutidos. Funções começam com **Benchmark** e recebem ***testing.B**.

```
func BenchmarkValidarTitulo(b *testing.B) {  
  
    for i := 0; i < b.N; i++ {  
  
        validarTitulo("Aprender Go")  
  
    }  
  
}
```

```
go test -bench=. -benchmem ./...
```

```
# BenchmarkValidarTitulo-8 1000000000 0.29 ns/op 0 B/op 0 allocs/op
```

Fase 6 — Router, Framework e Configuração

Objetivo: trocar o **net/http** padrão por um router mais robusto, implementar graceful shutdown, e centralizar configuração.

6.1 — Chi Router

O que você aprende: **chi** é o router mais popular do Go — leve, compatível com **net/http**, e adiciona middleware chain, grupos de rotas, e path params sem mágica.

```
go get github.com/go-chi/chi/v5

go get github.com/go-chi/chi/v5/middleware

package main

import (

    "net/http"

    "time"

    "github.com/go-chi/chi/v5"

    "github.com/go-chi/chi/v5/middleware"

)

func main() {

    r := chi.NewRouter()

    // Middleware global (aplica a todas as rotas)

    r.Use(middleware.Logger)           // Log de requests

    r.Use(middleware.Recoverer)       // Não crasha em panic

    r.Use(middleware.Timeout(30 * time.Second)) // Timeout global

    // Grupo de rotas com prefixo

    r.Route("/api/v1", func(r chi.Router) {

        r.Route("/tarefas", func(r chi.Router) {

            r.Get("/", h.Listar)        // GET /api/v1/tarefas

            r.Post("/", h.Criar)        // POST /api/v1/tarefas

            r.Get("/{id}", h.Buscar)    // GET /api/v1/tarefas/123

            r.Put("/{id}", h.Atualizar) // PUT /api/v1/tarefas/123
```

```

        r.Delete("/{id}", h.Deletar) // DELETE /api/v1/tarefas/123

    })

})

http.ListenAndServe(":8080", r)

}

```

Chi vs net/http vs Gin:

- `net/http` (stdlib) — suficiente pra projetos simples, mas verbose pra APIs grandes
- `chi` — leve, 100% compatível com `net/http`, ótimo pra APIs REST (recomendado)
- `gin` — mais opinado, maior ecossistema, mas cria acoplamento (binding próprio, context próprio)

6.2 — Configuração com Variáveis de Ambiente

O que você aprende: configuração em Go não usa `.env` automático como Laravel. O padrão é variáveis de ambiente diretamente, com uma struct de config.

```

// config/config.go

package config

import (

    "fmt"

    "os"

    "strconv"

)

type Config struct {

    Port    string

```

```

    DatabaseURL string

    LogLevel    string

    MaxWorkers int
}

func Carregar() (*Config, error) {

    port := os.Getenv("PORT")

    if port == "" {

        port = "8080"

    }

    dbURL := os.Getenv("DATABASE_URL")

    if dbURL == "" {

        return nil, fmt.Errorf("DATABASE_URL é obrigatória")

    }

    logLevel := os.Getenv("LOG_LEVEL")

    if logLevel == "" {

        logLevel = "info"

    }

    maxWorkers := 5

    if mw := os.Getenv("MAX_WORKERS"); mw != "" {

        var err error

        maxWorkers, err = strconv.Atoi(mw)

        if err != nil {

```

```
        return nil, fmt.Errorf("MAX_WORKERS inválido: %w", err)
    }
}

return &Config{
    Port:    port,
    DatabaseURL: dbURL,
    LogLevel: logLevel,
    MaxWorkers: maxWorkers,
}, nil
}
```

6.3 — Graceful Shutdown

O que você aprende: desligar o servidor sem cortar requests em andamento. Em produção, isso evita perda de dados quando o deploy atualiza a versão.

```
package main
```

```
import (
    "context"
    "fmt"
    "log"
    "net/http"
    "os"
    "os/signal"
```



```
"syscall"

"time"

)

func main() {

    // ... setup de router, db, etc ...

    srv := &http.Server{

        Addr:    ":8080",

        Handler: router,

    }

    // Canal pra receber sinais do OS (Ctrl+C, SIGTERM do Kubernetes)

    quit := make(chan os.Signal, 1)

    signal.Notify(quit, syscall.SIGINT, syscall.SIGTERM)

    // Iniciar servidor em goroutine

    go func() {

        fmt.Println("Servidor rodando em :8080")

        if err := srv.ListenAndServe(); err != http.ErrServerClosed {

            log.Fatal(err)

        }

    }()

    // Bloqueia até receber sinal

    <-quit
```

```
fmt.Println("Desligando servidor...")

// Dá 10 segundos pra requests em andamento terminarem

ctx, cancel := context.WithTimeout(context.Background(), 10*time.Second)

defer cancel()

if err := srv.Shutdown(ctx); err != nil {

    log.Fatal("Erro ao desligar:", err)

}

fmt.Println("Servidor desligado com sucesso")

}
```

Fase 7 — Docker e Deploy

Objetivo: containerizar o `taskctl` com Docker, rodar com `docker-compose` (API + PostgreSQL), e entender o build otimizado do Go.

7.1 — Dockerfile Multi-Stage

O que você aprende: Go compila pra um binário estático — o container final não precisa de runtime (diferente de PHP que precisa do php-fpm, ou Python que precisa do interpretador). Isso resulta em imagens Docker absurdamente pequenas.

```
# ---- Estágio 1: Build ----
```

```
FROM golang:1.22-alpine AS builder
```

```
WORKDIR /app
```

```
COPY go.mod go.sum ./
```

```
RUN go mod download          # Cache de dependências
```

COPY . .

CGO_ENABLED=0 = binário 100% estático (sem dependência de libc)

RUN CGO_ENABLED=0 GOOS=linux go build -o taskctl .

---- Estágio 2: Imagem final ----

FROM alpine:3.19

RUN apk --no-cache add ca-certificates # Certificados SSL

WORKDIR /app

COPY --from=builder /app/taskctl . # Só o binário

EXPOSE 8080

CMD ["/taskctl"]

Comparação de tamanho de imagem: | Stack | Imagem base | Imagem final típica | |---|---|---| |
PHP + Laravel | php:8.2-fpm | ~500MB | | Python + FastAPI | python:3.11-slim | ~200MB | | Go
(multi-stage) | alpine:3.19 | ~15-20MB |

O binário Go é autocontido — não precisa de interpretador, runtime, ou gerenciador de pacotes.

7.2 — Docker Compose (API + PostgreSQL)

O que você aprende: orquestrar múltiplos containers localmente. Equivalente ao `docker-compose` que você provavelmente já usa no GOINFRA.

docker-compose.yml

version: "3.9"

services:

db:

image: postgres:16-alpine

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

POSTGRES_DB: taskctl

ports:

- "5432:5432"

volumes:

- pgdata:/var/lib/postgresql/data

- ./migrations:/docker-entrypoint-initdb.d # Roda migrations na criação

api:

build: .

ports:

- "8080:8080"

environment:

DATABASE_URL: postgres://postgres:postgres@db:5432/taskctl?sslmode=disable

PORT: "8080"

depends_on:

- db

restart: unless-stopped

volumes:

pgdata:

docker-compose up -d # Sobe tudo

```
docker-compose logs -f api    # Acompanhar logs
```

```
docker-compose down          # Parar tudo
```

```
docker-compose down -v       # Parar e apagar dados do banco
```

7.3 — Makefile (Automação de Comandos)

O que você aprende: projetos Go usam **Makefile** pra centralizar comandos comuns. É convenção no ecossistema.

```
.PHONY: build run test lint docker-up docker-down
```

build:

```
    go build -o bin/taskctl .
```

run:

```
    go run .
```

test:

```
    go test -v -cover ./...
```

lint:

```
    golangci-lint run ./...
```

docker-up:

```
    docker-compose up -d --build
```

docker-down:

```
    docker-compose down
```

migrate:

```
    psql $(DATABASE_URL) < migrations/001_criar_tarefas.sql
```

Fase 8 — gRPC e Protobuf

Objetivo: aprender gRPC — protocolo de comunicação entre serviços mais eficiente que REST. É diferencial nas vagas (a CodiLime pede como desejável, e muitas outras também). Protobuf é o formato de serialização usado pelo gRPC.

8.1 — O que é gRPC e Por Que Importa

O que você aprende: gRPC é um protocolo de chamada remota (RPC) do Google. Mais rápido que REST+JSON porque usa HTTP/2 e Protobuf (binário, não texto). Usado pra comunicação entre microserviços internos.

Quando usar REST vs gRPC:

- **REST** → APIs públicas, frontends, terceiros (JSON é universal)
- **gRPC** → comunicação entre serviços internos (mais rápido, tipagem forte, streaming)

REST: Cliente → JSON → HTTP/1.1 → Servidor → JSON parse → processo

gRPC: Cliente → Protobuf → HTTP/2 → Servidor → Protobuf decode → processo

(binário, (multiplexado, (tipado,
compacto) mais rápido) sem parsing)

8.2 — Definindo o Serviço com Protobuf

O que você aprende: você define o contrato (API) em um arquivo `.proto`, e o compilador `protoc` gera código Go automaticamente.

Instalar protoc e plugins Go

```
go install google.golang.org/protobuf/cmd/protoc-gen-go@latest
```

```
go install google.golang.org/grpc/cmd/protoc-gen-go-grpc@latest
```

```
// proto/tarefa.proto

syntax = "proto3";

package taskctl;

option go_package = "github.com/aerlon/taskctl/proto";

// Mensagens (equivalente a structs/DTOs)

message Tarefa {

    int32 id = 1;

    string titulo = 2;

    bool feita = 3;

    string criada_em = 4;

}

message CriarTarefaRequest {

    string titulo = 1;

}

message ListarTarefasRequest {}

message ListarTarefasResponse {

    repeated Tarefa tarefas = 1; // repeated = array/slice

}

message TarefaIDRequest {

    int32 id = 1;

}

message Vazio {}
```

// Serviço (equivalente às rotas da API REST)

```
service TarefaService {  
  
    rpc Listar(ListarTarefasRequest) returns (ListarTarefasResponse);  
  
    rpc Buscar(TarefaIDRequest) returns (Tarefa);  
  
    rpc Criar(CriarTarefaRequest) returns (Tarefa);  
  
    rpc Deletar(TarefaIDRequest) returns (Vazio);  
  
}
```

Gerar código Go a partir do .proto

```
protoc --go_out=. --go-grpc_out=. proto/tarefa.proto
```

Isso gera dois arquivos: `tarefa.pb.go` (structs) e `tarefa_grpc.pb.go` (interface do serviço).

8.3 — Implementando o Servidor gRPC

O que você aprende: implementar a interface gerada pelo protoc e subir o servidor gRPC.

```
// grpc/server.go
```

```
package grpc
```

```
import (
```

```
    "context"
```

```
    pb "github.com/aerlon/taskctl/proto"
```

```
    "github.com/aerlon/taskctl/repository"
```

```
)
```

```
type TarefaGRPCServer struct {
```


pb.UnimplementedTarefaServiceServer // Necessário — permite adicionar métodos gradualmente

repo *repository.TarefaRepository

}

func NovoServer(repo *repository.TarefaRepository) *TarefaGRPCServer {

return &TarefaGRPCServer{repo: repo}

}

func (s *TarefaGRPCServer) Listar(ctx context.Context, req *pb.ListarTarefasRequest) (*pb.ListarTarefasResponse, error) {

tarefas, err := s.repo.Listar(ctx)

if err != nil {

return nil, err

}

var pbTarefas []*pb.Tarefa

for _, t := range tarefas {

pbTarefas = append(pbTarefas, &pb.Tarefa{

Id: int32(t.ID),

Titulo: t.Titulo,

Feita: t.Feita,

CriadaEm: t.CriadaEm.String(),

})

}

```

        return &pb.ListarTarefasResponse{Tarefas: pbTarefas}, nil
    }

func (s *TarefaGRPCServer) Criar(ctx context.Context, req *pb.CriarTarefaRequest) (*pb.Tarefa,
error) {

    tarefa, err := s.repo.Criar(ctx, req.Titulo)

    if err != nil {

        return nil, err

    }

    return &pb.Tarefa{

        Id:    int32(tarefa.ID),

        Titulo: tarefa.Titulo,

        Feita: tarefa.Feita,

        CriadaEm: tarefa.CriadaEm.String(),

    }, nil

}

// main_grpc.go — iniciar o servidor gRPC

package main

import (

    "fmt"

    "log"

    "net"

    "google.golang.org/grpc"

```

```

pb "github.com/aerlon/taskctl/proto"

grpcServer "github.com/aerlon/taskctl/grpc"

)

func main() {

    // ... setup de db e repo ...

    lis, err := net.Listen("tcp", ":50051")

    if err != nil {

        log.Fatal(err)

    }

    s := grpc.NewServer()

    pb.RegisterTarefaServiceServer(s, grpcServer.NovoServer(repo))

    fmt.Println("gRPC server rodando em :50051")

    log.Fatal(s.Serve(lis))

}

```

Testando com grpcurl (equivalente ao curl pra gRPC):

Instalar: `go install github.com/fullstorydev/grpcurl/cmd/grpcurl@latest`

`grpcurl -plaintext localhost:50051 taskctl.TarefaService/Listar`

`grpcurl -plaintext -d '{"titulo":"Aprender gRPC"}' \`

`localhost:50051 taskctl.TarefaService/Criar`

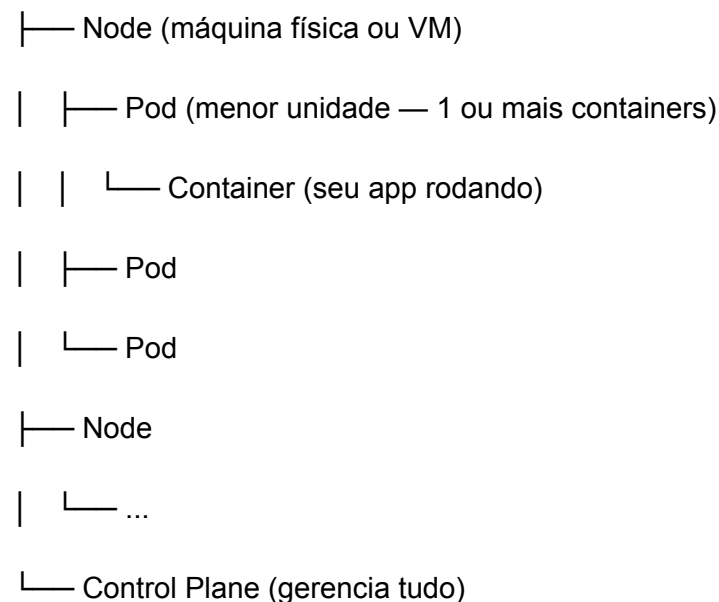
Fase 9 — Kubernetes Básico

Objetivo: entender como deployar o `taskctl` num cluster Kubernetes. Não precisa dominar K8s inteiro — precisa saber o suficiente pra conversar com DevOps e entender o ambiente onde seu código roda.

9.1 — Conceitos Fundamentais

O que você aprende: o vocabulário do Kubernetes e como as peças se encaixam.

Cluster K8s



Conceitos essenciais (o que cai em entrevista):

- **Pod** — wrapper de 1 ou mais containers. Efêmero (pode morrer e renascer a qualquer momento).
- **Deployment** — declara "quero N réplicas do meu pod rodando". K8s mantém automaticamente.
- **Service** — endpoint estável pra acessar pods (pods mudam de IP, o service não).
- **ConfigMap** — variáveis de configuração (equivalente ao `.env`, mas gerenciado pelo K8s).
- **Secret** — dados sensíveis (senhas, tokens) criptografados.
- **Ingress** — roteamento HTTP externo (equivalente ao Nginx reverse proxy).

9.2 — Manifests do taskctl

O que você aprende: como escrever os YAMLS que descrevem seu deploy no Kubernetes.

```
# k8s/deployment.yaml

apiVersion: apps/v1

kind: Deployment

metadata:

  name: taskctl-api

  labels:

    app: taskctl

spec:

  replicas: 3          # 3 instâncias rodando

  selector:

    matchLabels:

      app: taskctl

  template:

    metadata:

      labels:

        app: taskctl

    spec:

      containers:

        - name: taskctl
```

image: aerlon/taskctl:latest

ports:

- containerPort: 8080

env:

- name: DATABASE_URL

valueFrom:

secretKeyRef:

name: taskctl-secrets

key: database-url

- name: PORT

value: "8080"

resources:

requests: # Mínimo garantido

memory: "64Mi"

cpu: "50m"

limits: # Máximo permitido

memory: "128Mi"

cpu: "200m"

livenessProbe: # K8s reinicia se falhar

httpGet:

path: /health

port: 8080

initialDelaySeconds: 5

periodSeconds: 10

readinessProbe: # Só recebe tráfego quando pronto

httpGet:

path: /health

port: 8080

initialDelaySeconds: 3

periodSeconds: 5

k8s/service.yaml

apiVersion: v1

kind: Service

metadata:

name: taskctl-service

spec:

selector:

app: taskctl

ports:

- port: 80 # Porta exposta

targetPort: 8080 # Porta do container

type: ClusterIP # Interno (LoadBalancer pra expor externamente)

k8s/configmap.yaml

apiVersion: v1

kind: ConfigMap

metadata:

name: taskctl-config

data:

LOG_LEVEL: "info"

MAX_WORKERS: "5"

k8s/secret.yaml (valores em base64)

apiVersion: v1

kind: Secret

metadata:

name: taskctl-secrets

type: Opaque

data:

database-url: cG9zdGdyZXM6Ly9wb3N0Z3Jlc0BkYjo1NDMyL3Rhc2tjdGw=

base64 de: postgres://postgres:postgres@db:5432/taskctl

9.3 — Comandos Essenciais do kubectl

O que você aprende: os comandos do dia a dia com Kubernetes.

Aplicar manifests

kubectl apply -f k8s/

Ver pods rodando

kubectl get pods

# NAME	READY	STATUS	RESTARTS	AGE
--------	-------	--------	----------	-----

# taskctl-api-7d8f9b6c4-abc12	1/1	Running	0	5m
-------------------------------	-----	---------	---	----

# taskctl-api-7d8f9b6c4-def34	1/1	Running	0	5m
-------------------------------	-----	---------	---	----

# taskctl-api-7d8f9b6c4-ghi56	1/1	Running	0	5m
-------------------------------	-----	---------	---	----

Ver logs de um pod

kubectl logs taskctl-api-7d8f9b6c4-abc12

Acompanhar logs em tempo real

kubectl logs -f taskctl-api-7d8f9b6c4-abc12

Entrar no container (debug)

kubectl exec -it taskctl-api-7d8f9b6c4-abc12 -- /bin/sh

Escalar replicas

kubectl scale deployment taskctl-api --replicas=5

Ver detalhes de um recurso

kubectl describe deployment taskctl-api

Port-forward pra testar local

kubectl port-forward svc/taskctl-service 8080:80

9.4 — Horizontal Pod Autoscaler (HPA)

O que você aprende: escalar automaticamente baseado em CPU ou memória.

k8s/hpa.yaml

apiVersion: autoscaling/v2

kind: HorizontalPodAutoscaler

metadata:

name: taskctl-hpa

spec:

scaleTargetRef:

apiVersion: apps/v1

kind: Deployment

name: taskctl-api

minReplicas: 2

maxReplicas: 10

metrics:

- type: Resource

resource:

name: cpu

target:

type: Utilization

averageUtilization: 70 # Escala quando CPU > 70%

Pra praticar sem cluster real: use `minikube` ou `kind` (Kubernetes in Docker) localmente.

Instalar kind

go install sigs.k8s.io/kind@latest

```
kind create cluster --name taskctl
```

```
kubectl apply -f k8s/
```

Fase 10 — Projeto Final Completo (Portfólio)

Objetivo: adicionar tudo que falta pra o `taskctl` ser um projeto de portfólio real — autenticação, rate limiting, observabilidade, CI/CD. É o que diferencia "fiz um CRUD" de "construí um sistema de produção".

10.1 — Autenticação com JWT

O que você aprende: implementar autenticação stateless via JWT (JSON Web Token) — padrão em APIs REST.

```
go get github.com/golang-jwt/jwt/v5
```

```
// auth/jwt.go
```

```
package auth
```

```
import (
```

```
    "errors"
```

```
    "time"
```

```
    "github.com/golang-jwt/jwt/v5"
```

```
)
```

```
var jwtSecret = []byte("trocar-em-producao") // Em produção: variável de ambiente
```

```
type Claims struct {
```

```
    UserID int    `json:"user_id"`
```

```
    Email string `json:"email"`
```

```

    jwt.RegisteredClaims
}

func GerarToken(userID int, email string) (string, error) {

    claims := Claims{

        UserID: userID,

        Email: email,

        RegisteredClaims: jwt.RegisteredClaims{

            ExpiresAt: jwt.NewNumericDate(time.Now().Add(24 * time.Hour)),

            IssuedAt: jwt.NewNumericDate(time.Now()),

        },

    }

    token := jwt.NewWithClaims(jwt.SigningMethodHS256, claims)

    return token.SignedString(jwtSecret)

}

func ValidarToken(tokenStr string) (*Claims, error) {

    token, err := jwt.ParseWithClaims(tokenStr, &Claims{},

        func(t *jwt.Token) (interface{}, error) {

            return jwtSecret, nil

        })

    if err != nil {

        return nil, err

    }

```

```

claims, ok := token.Claims.(*Claims)

if !ok || !token.Valid {

    return nil, errors.New("token inválido")

}

return claims, nil

}

// Middleware de autenticação

func AuthMiddleware(next http.Handler) http.Handler {

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {

        header := r.Header.Get("Authorization")

        if header == "" {

            http.Error(w, `{"erro":"token ausente"}`, http.StatusUnauthorized)

            return

        }

        // Formato: "Bearer <token>"

        tokenStr := strings.TrimPrefix(header, "Bearer ")

        claims, err := auth.ValidarToken(tokenStr)

        if err != nil {

            http.Error(w, `{"erro":"token inválido"}`, http.StatusUnauthorized)

            return

        }

```

```

    // Adicionar claims ao context da request

    ctx := context.WithValue(r.Context(), "user_id", claims.UserID)

    next.ServeHTTP(w, r.WithContext(ctx))

})

}

// No router (Chi):

r.Route("/api/v1", func(r chi.Router) {

    r.Post("/login", loginHandler)    // Público

    r.Group(func(r chi.Router) {

        r.Use(AuthMiddleware)        // Protegido

        r.Get("/tarefas", h.Listar)

        r.Post("/tarefas", h.Criar)

    })

})

```

10.2 — Rate Limiting

O que você aprende: proteger a API contra abuso limitando requests por IP ou por usuário.

```

// middleware/ratelimit.go

package middleware

import (

    "net/http"

    "sync"

```

```

    "time"
)

type RateLimiter struct {

    mu    sync.Mutex

    visitors map[string]*visitor

    limit  int

    window time.Duration

}

type visitor struct {

    count  int

    lastSeen time.Time

}

func NovoRateLimiter(limit int, window time.Duration) *RateLimiter {

    rl := &RateLimiter{

        visitors: make(map[string]*visitor),

        limit:    limit,

        window:   window,

    }

    // Limpar visitantes antigos periodicamente

    go rl.limpar()

    return rl

```

```
}
```

```
func (rl *RateLimiter) limpar() {
```

```
    for {
```

```
        time.Sleep(rl.window)
```

```
        rl.mu.Lock()
```

```
        for ip, v := range rl.visitors {
```

```
            if time.Since(v.lastSeen) > rl.window {
```

```
                delete(rl.visitors, ip)
```

```
            }
```

```
        }
```

```
        rl.mu.Unlock()
```

```
    }
```

```
}
```

```
func (rl *RateLimiter) Middleware(next http.Handler) http.Handler {
```

```
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
```

```
        ip := r.RemoteAddr
```

```
        rl.mu.Lock()
```

```
        v, existe := rl.visitors[ip]
```

```
        if !existe || time.Since(v.lastSeen) > rl.window {
```

```
            rl.visitors[ip] = &visitor{count: 1, lastSeen: time.Now()}
```

```
            rl.mu.Unlock()
```

```
            next.ServeHTTP(w, r)
```



```
        return

    }

    v.count++

    v.lastSeen = time.Now()

    if v.count > rl.limit {

        rl.mu.Unlock()

        http.Error(w, `{"erro":"rate limit excedido"}`, http.StatusTooManyRequests)

        return

    }

    rl.mu.Unlock()

    next.ServeHTTP(w, r)

}))

}
```

10.3 — Logging Estruturado com slog

O que você aprende: Go 1.21+ tem o pacote `log/slog` embutido pra logging estruturado (JSON). Em produção, logs estruturados são obrigatórios (Datadog, Grafana, ELK precisam de JSON).

```
package main
```

```
import (
```

```
    "log/slog"
```

```
    "net/http"
```

```

"os"

"time"

)

func main() {

    // Logger JSON pra produção

    logger := slog.New(slog.NewJSONHandler(os.Stdout, &slog.HandlerOptions{

        Level: slog.LevelInfo,

    })))

    slog.SetDefault(logger)

    slog.Info("servidor iniciado",

        "port", 8080,

        "environment", "production",

    )

    // Output: {"time":"2026-08-10T...", "level":"INFO", "msg":"servidor
    iniciado", "port":8080, "environment":"production"}

}

// Middleware de logging estruturado

func LogMiddleware(next http.Handler) http.Handler {

    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {

        start := time.Now()

        next.ServeHTTP(w, r)

        slog.Info("request",

```

```

    "method", r.Method,

    "path", r.URL.Path,

    "duration_ms", time.Since(start).Milliseconds(),

    "remote_addr", r.RemoteAddr,

)

})

}

```

10.4 — Health Check e Readiness

O que você aprende: endpoints que Kubernetes (e qualquer orquestrador) usa pra saber se seu app tá vivo e pronto.

```

func healthHandler(pool *pgxpool.Pool) http.HandlerFunc {

    return func(w http.ResponseWriter, r *http.Request) {

        w.Header().Set("Content-Type", "application/json")

        // Verificar conexão com o banco

        err := pool.Ping(r.Context())

        if err != nil {

            w.WriteHeader(http.StatusServiceUnavailable)

            fmt.Fprintf(w, `{"status":"unhealthy","db":"down","error":"%s"}`, err)

            return

        }

        fmt.Fprintf(w, `{"status":"healthy","db":"up"}`)
    }
}

```

```
}  
  
}  
  
// No router:  
  
mux.HandleFunc("GET /health", healthHandler(pool))
```

10.5 — CI/CD com GitHub Actions

O que você aprende: pipeline automatizado que roda testes, builda a imagem Docker, e faz push pro registry a cada commit.

```
# .github/workflows/ci.yml
```

```
name: CI/CD
```

```
on:
```

```
  push:
```

```
    branches: [main]
```

```
  pull_request:
```

```
    branches: [main]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    services:
```

```
      postgres:
```

```
        image: postgres:16
```

```
        env:
```

POSTGRES_USER: postgres

POSTGRES_PASSWORD: postgres

POSTGRES_DB: taskctl_test

ports:

- 5432:5432

options: >-

--health-cmd pg_isready

--health-interval 10s

--health-timeout 5s

--health-retries 5

steps:

- uses: actions/checkout@v4

- uses: actions/setup-go@v5

with:

go-version: "1.22"

- run: go test -v -cover ./...

env:

DATABASE_URL:

postgres://postgres:postgres@localhost:5432/taskctl_test?sslmode=disable

build:

needs: test

runs-on: ubuntu-latest

```
if: github.ref == 'refs/heads/main'

steps:

  - uses: actions/checkout@v4

  - uses: docker/login-action@v3

    with:

      username: ${ secrets.DOCKER_USER }

      password: ${ secrets.DOCKER_TOKEN }

  - uses: docker/build-push-action@v5

    with:

      push: true

      tags: aerlon/taskctl:latest,aerlon/taskctl:${ github.sha }
```

10.6 — Estrutura Final do Projeto

taskctl/

```
|— .github/
|  └─ workflows/
|     └─ ci.yml
|— auth/
|  └─ jwt.go
|— config/
|  └─ config.go
|— db/
```

```
|   └─ postgres.go

└─ grpc/

|   └─ server.go

└─ handler/

|   └─ tarefa_handler.go

|   └─ tarefa_handler_test.go

└─ middleware/

|   └─ auth.go

|   └─ logging.go

|   └─ ratelimit.go

└─ migrations/

|   └─ 001_criar_tarefas.sql

└─ proto/

|   └─ tarefa.proto

|   └─ tarefa.pb.go

|   └─ tarefa_grpc.pb.go

└─ repository/

|   └─ tarefa_repo.go

|   └─ tarefa_repo_test.go

└─ k8s/

|   └─ deployment.yaml
```

- | |— service.yaml
- | |— configmap.yaml
- | |— secret.yaml
- | |— hpa.yaml
- |— docker-compose.yml
- |— Dockerfile
- |— Makefile
- |— go.mod
- |— go.sum
- |— main.go
- |— README.md

Checklist de Skills por Fase

Fase	Skills adquiridas	Relevância pra vaga
1	Sintaxe, tipos, funções, structs, interfaces, erros	Base obrigatória
2	HTTP server, JSON, CRUD, middleware	Qualquer vaga backend
3	PostgreSQL, pgx, repository pattern, migrations	Qualquer vaga backend
4	Goroutines, channels, mutex, worker pools	Diferencial Go — o que todo entrevistador testa
5	Testes unitários, table-driven, httptest, mocks	Requisito de Pleno

Fase	Skills adquiridas	Relevância pra vaga
6	Chi router, config, graceful shutdown	Padrão de mercado
7	Docker multi-stage, compose, Makefile	Requisito universal
8	gRPC, Protobuf	Desejável em vagas Go (CodiLime pede)
9	Kubernetes: pods, deployments, services, HPA	Desejável, demonstra maturidade
10	JWT, rate limiting, logging, CI/CD	Completa o perfil de produção